



Módulo Profesional 12: Proyecto Común Arduino

Actividad 5 UF1

CICLO FORMATIVO DE GRADO MEDIO EN SMX

MODALIDAD PRESENCIAL

OBJETIVOS	Utilizar el Arduino (ESP32 Wrover) para aprender a manipular las funciones RGB que incluye.
RÚBRICA	1. Respuestas completas a las preguntas. Bien desarrolladas. (1pto) 2. Aspecto del documento (1pto) 3. Faltas de ortografía (-0.05ptos) 4. Nombre del documento en el mismo formato que lo recibes: M05UF1AX_NombreApellido.PDF 5. Documento en PDF. De no ser así no califico. 6. Por cada día fuera del plazo de entrega: -1pto. 7. Actividades iguales: 0pto 8. Webgrafía: 1pto Aleatoriamente se visitaran los puestos de trabajo, para que muestren la página del WIX y el desarrollo con Arduino, explicando los aspectos fundamentales del programa y sus funciones.
DEDICACIÓN	5 horas
REFERENCIA	Recursos de campus y bibliografía
Entrega Fecha	El enlace a una página de WIX 1/11/23

Introducción:

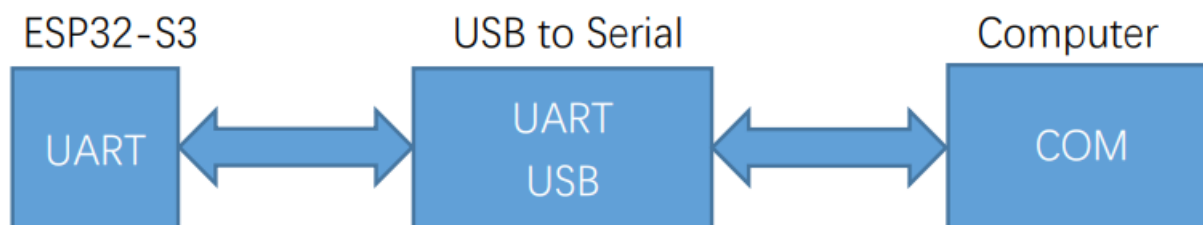
La comunicación serial es un método de transferencia de datos en el que la información se envía de manera secuencial, un bit a la vez, a través de un solo conductor o un par de conductores.

En otras actividades hemos visto como la comunicación serial nos permitía subir información a la placa ESP32. Hay varios conceptos que tenemos que tener claros para esta comunicación: tramas, baud rate y la sincronización.

La información se organiza en **tramas**. Cada trama suele comenzar con un bit de inicio y terminar con uno o más bits de parada. Estos bits ayudan al receptor a sincronizarse y a saber dónde comienza y termina cada trama.

La velocidad a la que se transmiten los bits se mide en baudios (**baud rate**). Representa la cantidad de cambios de estado (de 0 a 1 o de 1 a 0) por segundo. En Arduino se llama "Upload speed" y lo encontramos en "Herramientas". Por ejemplo, a una velocidad de 9600 baudios, se transmiten 9600 bits por segundo.

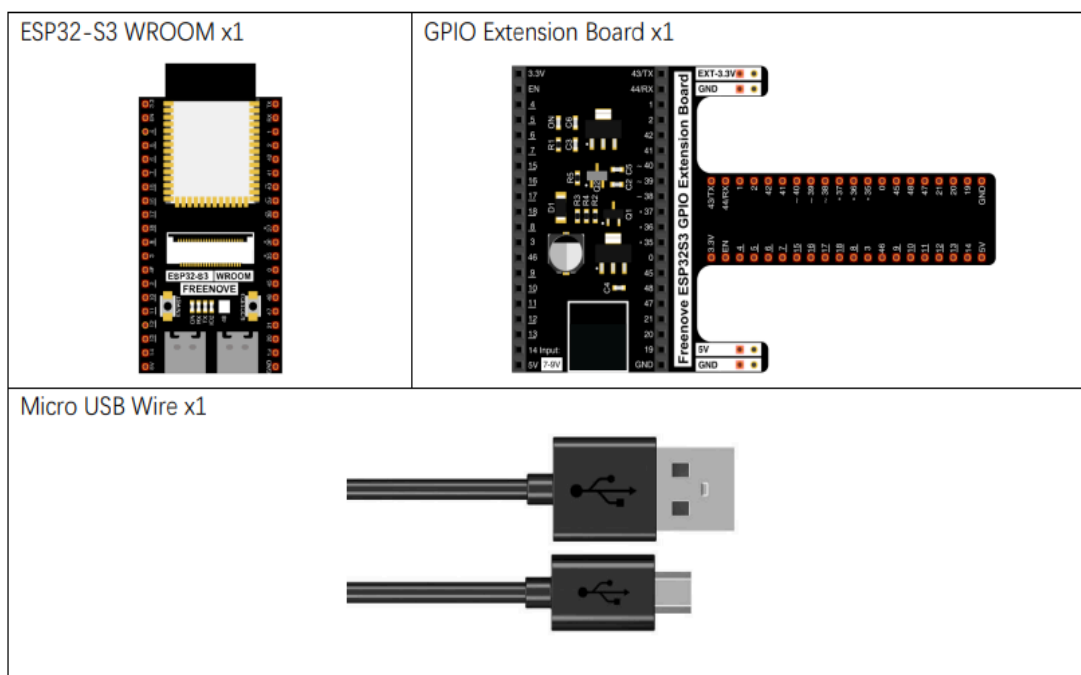
La **sincronización** es crucial en la comunicación serial, los dispositivos deben estar configurados para interpretar los bits en el momento correcto. La secuencia de bits y la velocidad deben ser conocidas y acordadas entre el transmisor y el receptor mediante protocolos. Algunos ejemplos son UART (Universal Asynchronous Receiver/Transmitter) y SPI (Serial Peripheral Interface).



Proyecto 6.1) Serial communication

Para esta práctica vamos simplemente a probar como funciona la comunicación de la placa con el ordenador para, en próximas prácticas, explotar esta funcionalidad.

¿Qué necesitamos para hacer el proyecto?



¿Qué tenemos que hacer?

Para poder observar la salida de información, debemos acudir a herramientas > Monitor serie y aparecerá una pestaña extra junto a la de “salida” por la que siempre leemos las incidencias con la placa.

Pon el monitor serial en “115200 baud” para que funcione.

Utiliza el siguiente código:

```
void setup() {
  Serial.begin(115200);
  Serial.println("ESP32S3 iniciada! ");
}

void loop() {
  Serial.printf("Tiempo desde ejecución: %.1f s\n", millis() / 1000.0f);
  delay(1000);
}
```

Preguntas:

A continuación, de realizar la práctica responde estas preguntas que deberás añadir en la información del WIX:

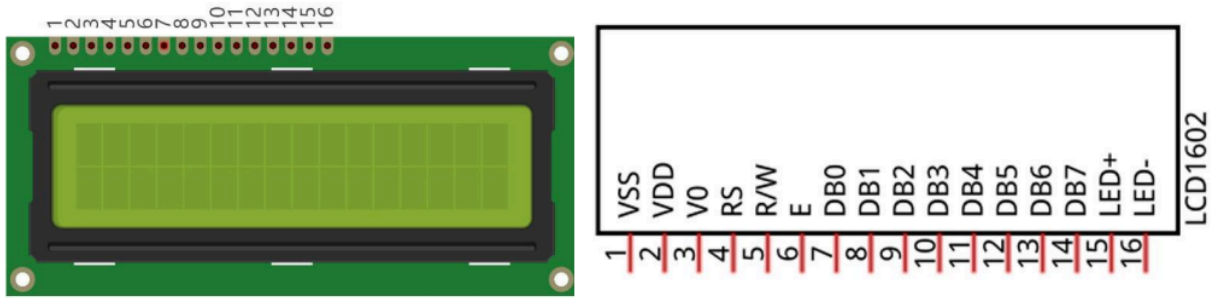
1. ¿Que aparece en serial monitor?
2. Pulsa los botones de boot+EN que hay en la placa de Arduino, ¿qué ocurre? Ahora pulsa solo EN, ¿qué ha ocurrido? ¿para qué nos puede servir esto?
Se iluminan más los cuadrados al apretar los 2 botones. Nos puede servir para cambiar el nivel de brillo , aplicando el comando que esta abajo y al subir el valor del comando iluminará mas .
3. ¿Qué indica la línea de código `Serial.begin(115200);`?
4. Averigua que significa `%.1f s\n`.

Sube las respuestas y los diseños en los apartados correspondientes de tu página al Wix.

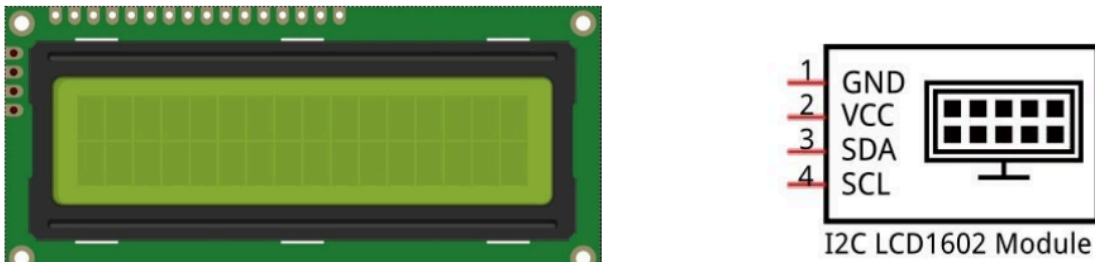
Proyecto 6.2) Panel LCD1602

¿Qué necesitamos para hacer el proyecto?

Una pantalla **LCD1602 típica** puede mostrar 2 líneas de caracteres en 16 columnas y es capaz de mostrar números, letras, símbolos, código ASCII, etc. A continuación, puedes ver los pines de los que dispone:



Como puedes ver son muchos pines para tener controlados así que **se simplifica en la versión I2C**, que conecta la entrada en serie y la salida en paralelo, lo cual nos permite usar solo 4 líneas para operar la pantalla:

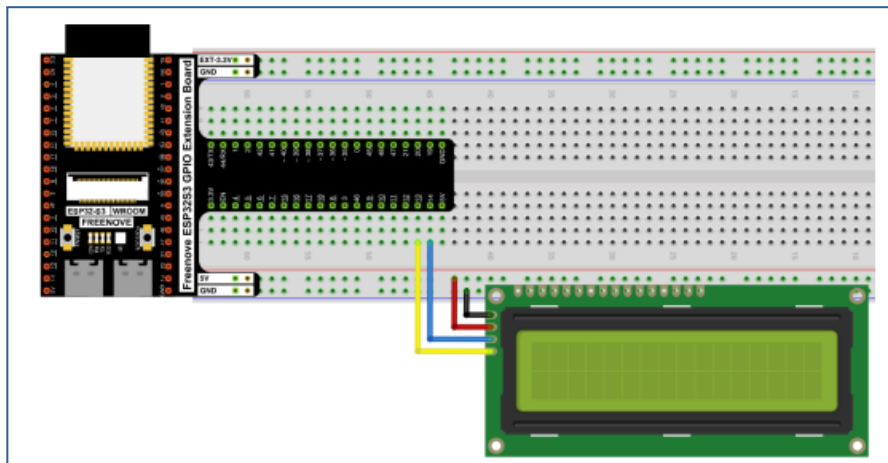
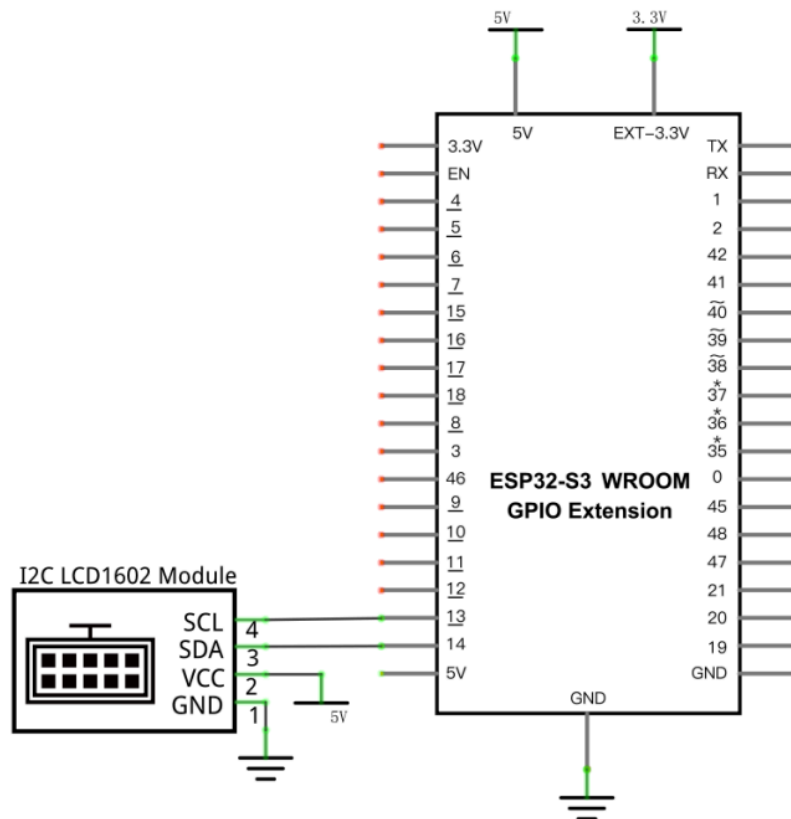


El chip IC de serie a paralelo utilizado en este módulo es PCF8574T (PCF8574AT), y **su dirección I2C predeterminada es 0x27(0x3F)**.

¿Qué necesitamos para hacer el proyecto?

Este es el circuito que debéis diseñar:

Schematic diagram



Como veis necesitáis:

4 jumpers de tipo hembra-macho

La pantalla LCD

¿Qué tenemos que hacer?

Revisa la información del capítulo 20 del libro:

1. Conecta la pantalla tal y como se muestra en el circuito anterior
2. En el Arduino IDE incluye la librería que puedes encontrar en [Github](#), busca el paquete de *LiquidCrystal_I2C.zip* y añádelo al IDE para que funcione.
3. Pon el "Upload speed" de "Herramientas>Upload speed" a 115200
4. NO utilices el puerto 12 de la GPIO
5. Usa el código que encuentres en el capítulo.

Preguntas:

- 1) Revisa las conexiones en el circuito eléctrico y explica para que se utiliza cada una:

SCL	Sirve para sincronizar la comunicación entre el ESP32 (maestro) y la pantalla LCD (esclavo).
SDA	Por aquí se envían los datos y comandos desde el ESP32 a la pantalla (texto, instrucciones). Funciona de forma bidireccional (puede enviar y recibir).
VCC	Es la alimentación del módulo LCD. Normalmente se conecta a 5V o 3.3V .
GND	Es la tierra o referencia eléctrica. Cierra el circuito eléctrico.

- 2) ¿Que hace la función "lcd.print()"? ¿Y "lcd.clear"?

Muestra **texto o números** en la pantalla LCD **a partir de la posición actual del cursor**.

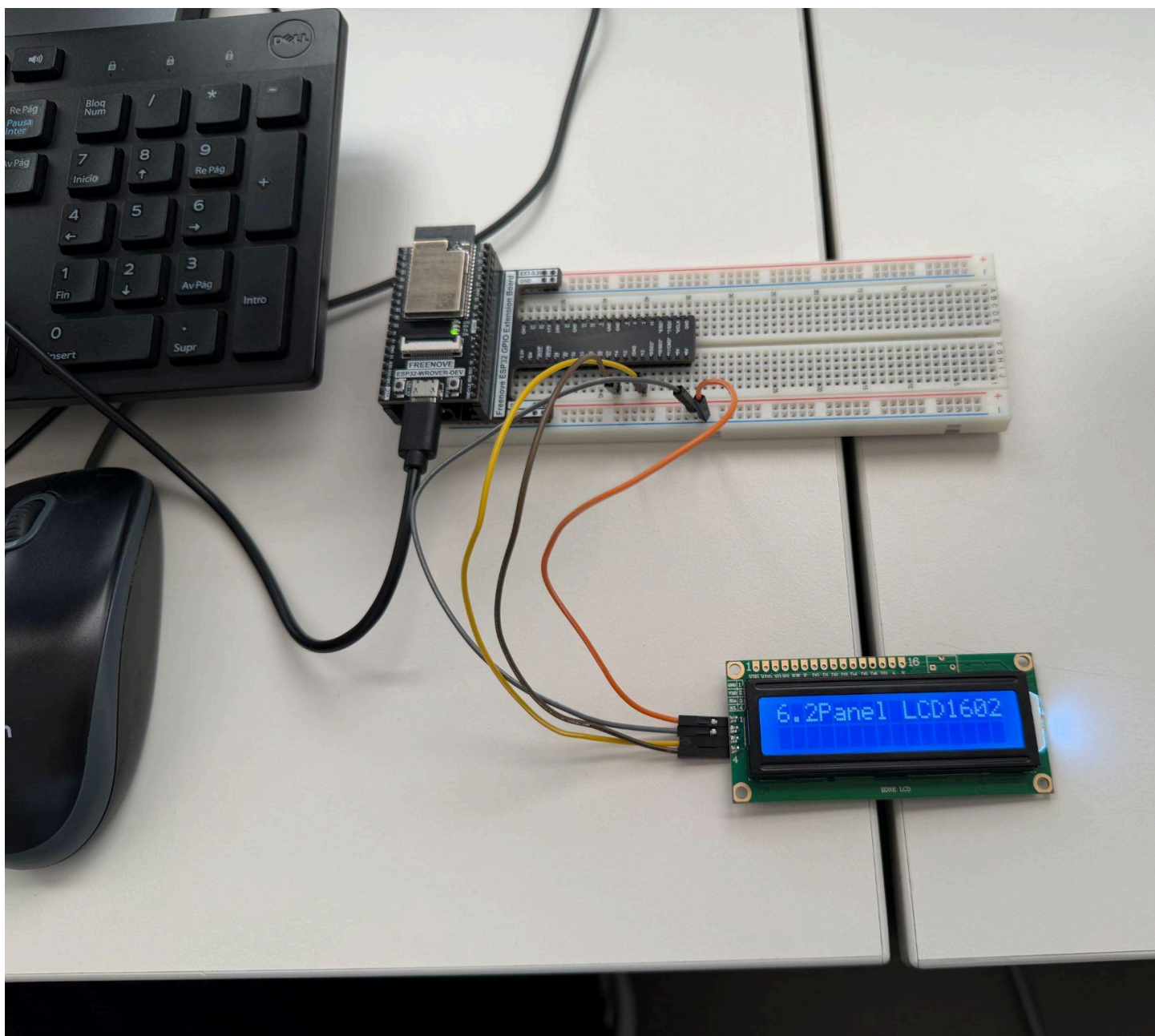
Es equivalente a un `Serial.print()`, pero en lugar del monitor serie, escribe en el LCD.

lcd .clear **Borra completamente** el contenido de la pantalla LCD.

Devuelve el cursor a la **posición inicial (0,0)**.

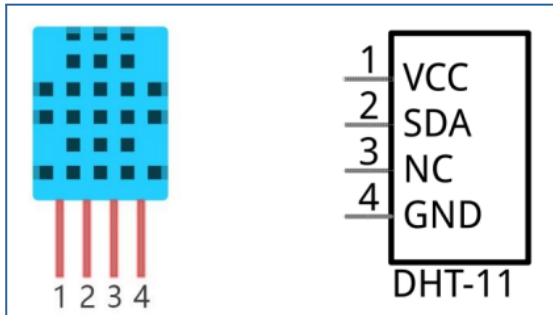
- 3) Por último, busca como conseguir que el mensaje de la primera fila se desplace de izquierda a derecha o a la inversa.

No olvides subir los ejemplos y las explicaciones al Wix así como todas sus respuestas.



Proyecto 6.3) Crear una estación meteorológica y recuperar los datos

¿Qué necesitamos para hacer el proyecto?



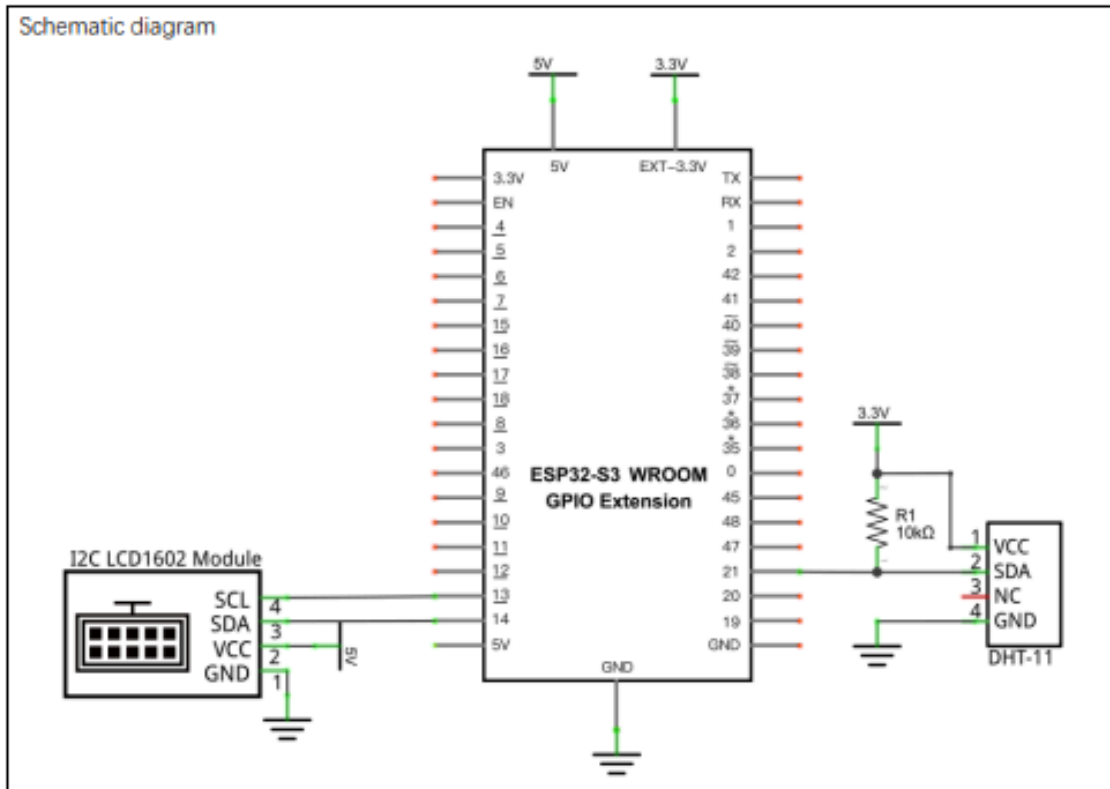
Un higrómetro es un instrumento de medición utilizado para registrar y monitorizar las variaciones de temperatura y humedad relativa en el tiempo.

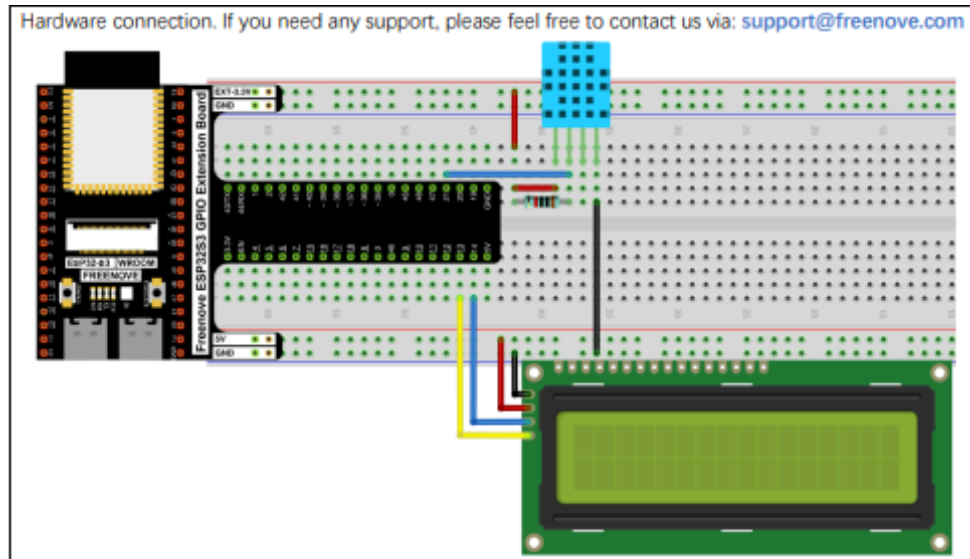
Su diseño combina las funciones de un **termómetro** (para medir la temperatura) y un **higrómetro** (para medir la humedad relativa).

En nuestro proyecto utilizaremos el dispositivo DHT11 que tiene 4 pines de los cuales el SDA es el que registra los datos por el pin, el VCC y el GND son los que le ofrecen la energía y cierran el circuito.

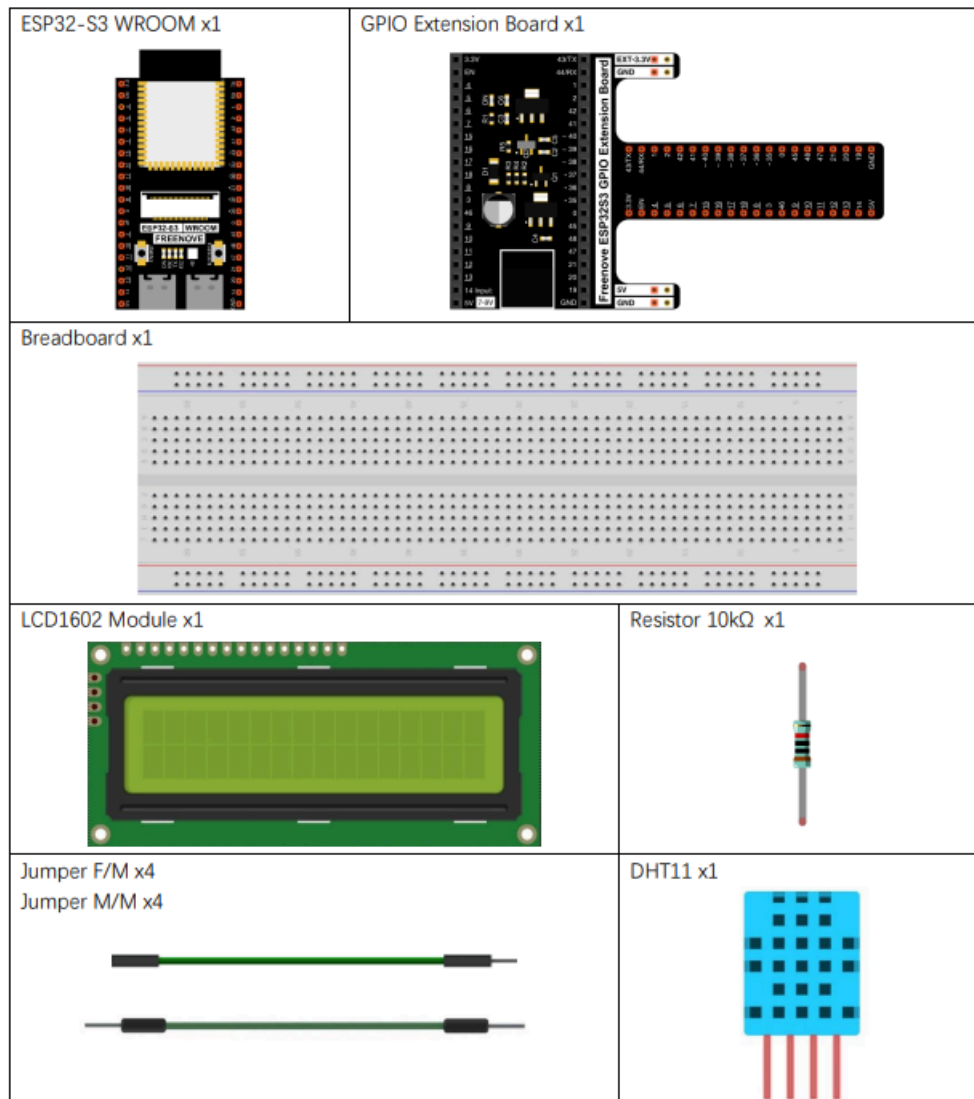
¿Qué necesitamos para hacer el proyecto?

Este es el circuito que debéis diseñar:





Y estos son los materiales:



¿Qué tenemos que hacer?

Revisa la información del capítulo 24 del libro:

1. Conecta la pantalla y el higrómetro tal y como se muestra en el circuito anterior
2. En el Arduino IDE incluye la librería que puedes encontrar en [Github](#), busca el paquete de **DHT_sensor_library_for_ESPx-1.17.0.zip** y añádelo al IDE para que funcione.
3. Pon el "Upload speed" de "Herramientas>Upload speed" a 115200
4. NO utilices el puerto 12 de la GPIO para ninguno de los pines
5. Usa el código que encuentres en el capítulo.

Preguntas:

- 1) Primero prueba que el código funciona por monitor serial. Prueba a soplar sobre el sensor para modificar los valores de humedad.



- 2) Ahora prueba a mostrar los valores en la pantalla LCD.
- 3) Busca que hace esta línea `DHTesp dht;` al principio del código. ¿Que es un objeto en programación y que es lo que hace?

La línea `DHTesp dht;` declara y crea un **objeto** llamado `dht` a partir de la clase `DHTesp`, que pertenece a la librería `DHTesp` y se usa para manejar sensores DHT (como el DHT11 o DHT22). En programación, un **objeto** es una instancia de una clase: combina **datos** y **funciones** que representan algo concreto; en este caso, el sensor de temperatura y humedad. Ese objeto `dht` se encarga de comunicarse con el sensor físico, inicializarlo y proporcionar métodos como `setup()` o

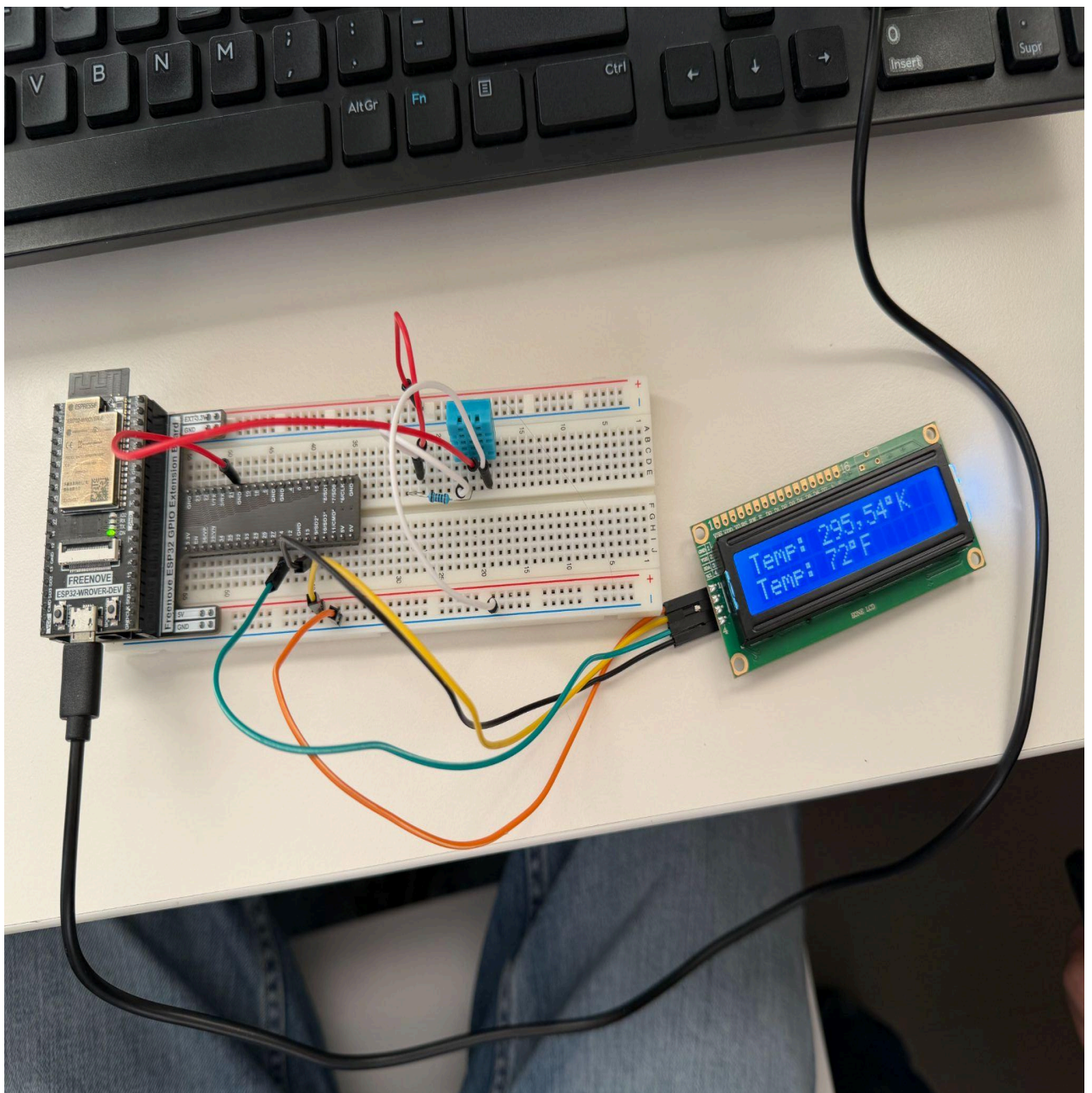
`getTempAndHumidity()` para leer sus valores sin que el programador tenga que manejar directamente los detalles eléctricos o de protocolo.

- 4) Prueba a codificar los valores para que muestre en la primera fila la temperatura en grados Kelvin y en la segunda fila en grados Farenheit.

Programalo de tal forma que se muestre algo así:

Temp: 303,15°K

Temp: 86°F



Sube las respuestas y los diseños en los apartados correspondientes de tu página al Wix.